

WhatsApp Photo Archive — Quickstart

A step-by-step guide that takes you from "I have an iPhone with WhatsApp" to "I can search every photo in my chat history from my laptop and my phone, anywhere in the world".

Time: ~30–45 minutes the first time. Re-builds later take ~5 minutes.

You'll end with: one self-contained, password-protected SQLite archive on your laptop, a small web viewer with a search box and thumbnails, and an encrypted Tailscale connection so the same viewer opens cleanly on your iPhone whether you're at home, on cellular, or anywhere else.

See it before you build it (optional)

If you want to preview the viewer UI before doing any of the iPhone extraction, install the tool (Step 3 below) and run:

```
wa-photos demo
```

This builds a fake 12-photo archive in `~/ .cache/whatsapp_photos/demo/` and opens the same viewer you'll use for your real data. When you're ready, switch to the real workflow below.

What you'll need

- A Mac or Windows PC (the "laptop")
 - Your iPhone with WhatsApp installed and chat history present
 - A USB / USB-C / Lightning cable
 - **iMazing** (free trial is enough to view; the export step needs a paid license). Alternatives: iExplorer, iBackup Viewer Pro
 - **Python 3.10 or newer** on the laptop
 - **Tailscale** account (free for personal use) — only required if you want phone access from outside your home WiFi
-

Step 1 — Make a fresh iPhone backup

1. Connect the iPhone to the laptop with the cable. Unlock the phone and tap Trust if prompted.
2. Open iMazing. The phone appears in the left sidebar.
3. Click the phone, then Back Up Now (top toolbar).
4. Wait for the backup to finish. If your iTunes / Finder backup is encrypted, iMazing asks for the backup password — enter it.

Tip: if you want a quicker rebuild later, leave "Automatic Backup" on so new chats are captured without manually re-running this step.

Step 2 — Extract WhatsApp data with iMazing

1. With your iPhone selected in iMazing, click Apps (some versions call it Manage Apps).

2. Find **WhatsApp Messenger** in the list.
3. Right-click it → Export App Data. Or use the Extract App Data button in the toolbar.
4. Choose a destination folder on the laptop, e.g. ~/wa-extract/.
5. Wait for export. This is the slowest step — five to thirty minutes depending on chat-history size and how many photos you have.

When it finishes, your destination folder will contain something like:

```
wa-extract/  
├─ ChatStorage.sqlite  
├─ Library/  
└─ Message/  
    └─ Media/  
        ├── 111@s.whatsapp.net/  
        ├── 222@s.whatsapp.net/  
        └─ ...
```

Both ChatStorage.sqlite and the Message/ tree must be present. The ingest tool needs both in the same parent directory.

Step 3 — Install the photo archive tool

Open a terminal (Mac: Terminal app; Windows: PowerShell).

```
git clone <repository-url>  
cd Whatsappmerger  
python3 -m pip install -e .
```

Quick sanity check:

```
wa-photos --version
```

If you get the version number back, you're ready.

Step 4 — Build the searchable archive

```
wa-photos ingest ~/wa-extract/ChatStorage.sqlite ~/wa-extract \  
-o photos.db \  
--password
```

When prompted, enter and confirm a strong password. This is the password your phone's browser will ask for later.

The tool prints a summary like:

```
Ingested -> photos.db  
  Photos inserted:      4 821  
  Chats:                78  
  Skipped (file missing): 12  
  Skipped (not image):  3
```

Elapsed:	47.21s
----------	--------

A few skipped rows are normal — those are messages whose attachment file isn't on disk anymore (e.g. expired media that was never saved).

Step 5 — Install Tailscale on laptop and iPhone

1. **Laptop:** download from tailscale.com/download → install → sign in (Google / Apple / Microsoft / GitHub account all fine).

2. **iPhone:** App Store → search Tailscale → install → sign in to the **same** account.

You can verify they're on the same tailnet by opening

<https://login.tailscale.com/admin/machines> — both devices should be listed.

No port-forwarding, no router configuration, no certificates.

Step 6 — Start the server

```
wa-photos serve photos.db --host 0.0.0.0
```

You'll see a banner like this:

```
Serving photo archive on http://0.0.0.0:8765/  
LAN: http://192.168.1.42:8765/  
Tailscale IPv4: http://100.84.17.3:8765/  
Tailscale MagicDNS: http://mylaptop.tail-abcd.ts.net:8765/  
(Password protection: ON – your browser will prompt for credentials.)
```

Copy the **Tailscale MagicDNS** URL — it's the one that works from anywhere.

Leave this terminal window open while you want the archive reachable. Closing it stops the server.

Step 7 — Open the archive on your phone

1. On the iPhone, open the Tailscale app and make sure the toggle at the top is **on**.

2. Open Safari (or Chrome) and paste the MagicDNS URL from Step 6.

3. iOS will prompt for credentials. Username can be anything (it's ignored); enter the password you set in Step 4.

4. You're in. Use the search box, the chat / sender filters, the date range, and tap any thumbnail to view full-size.

Bookmark the URL so you don't have to re-type it.

Tips & troubleshooting

- **MagicDNS URL doesn't resolve.** Open the Tailscale admin console → DNS → make sure MagicDNS is enabled. It's on by default for new tailnets.

-
- **A photo shows as broken.** iMazing didn't pull every media file. Run the export again with Include Media / Include Documents options checked, then re-run `wa-photos ingest`.
 - **Forgot the password.** No recovery — the password is hashed. Reset with `wa-photos password set photos.db`.
 - **Want to refresh with new messages.** Take a fresh iPhone backup, re-extract with iMazing, re-run `wa-photos ingest ...` with the same output path. The old `photos.db` is overwritten.
 - **Stop the server.** Press Ctrl-C in the terminal where it's running. Tailscale stays up; restart with the same command later.
 - **Want to share with another person.** Don't — the archive contains every photo you've ever exchanged on WhatsApp. Treat it like the sensitive personal data it is.
-

What stays private

- The archive (`photos.db`) and the original photos stay on your laptop. Nothing is uploaded.
 - The password is stored as a PBKDF2-HMAC-SHA256 hash with a per-archive random salt — not as plaintext.
 - Tailscale traffic is end-to-end encrypted via WireGuard. Your password is never sent in cleartext over any network.
 - The viewer makes no outbound connections. No telemetry, no analytics.
-

Generated by ``docs/build_pdf.py``. Edit ``docs/QUICKSTART.md`` and re-run the script to update the PDF.